Despliegue de una API Express en Vercel

Documentación de lo realizado en `usd-alfam-services` y checklist para repetirlo en otro repositorio.

1. Qué se hizo

Paso	Detalle	Quién
Correr en local	`npm start`. Arrancó en el puerto 8081 porque el `.env` definía `PORT` y pisa el valor por defecto 8085 de `index.js` (`process.env.PORT || 8085`).	Claude
Preparar el código	`index.js` exporta la app (`module.exports = app`) y solo hace `listen` si se ejecuta directo (`require.main === module`).	Claude
Archivos de configuración	Se agregaron `vercel.json`, `.vercelignore` y `engines.node >=20` en `package.json`.	Claude
Renombrar el proyecto	Nombre del paquete (`package.json` y `package-lock.json`) y link de docs en `index.js` pasaron a `usd-alfam-services`.	Claude
Importar en Vercel	vercel.com/new → Continue with GitHub → repo → Deploy.	Usuario
Verificar identidad	Vercel pidió un código de 6 dígitos por mail.	Usuario
Rama develop	Creada y subida con `git push -u origin develop`.	Claude
README	URL nueva, fuentes de datos correctas y sección «Despliegue» con el flujo de ramas.	Claude

2. Archivos de configuración

vercel.json

```
{
  "$schema": "https://openapi.vercel.sh/vercel.json",
  "version": 2,
  "builds": [{ "src": "index.js", "use": "@vercel/node" }],
  "routes": [{ "src": "/(.*)", "dest": "index.js" }]
}
```

.vercelignore

```
node_modules
.env
.git
*.http
.metadata
eslint.config.js
```

package.json (agregado)

```
"engines": { "node": ">=20" }
```

index.js (patrón clave)

```
const app = express()
// ... middlewares y rutas ...
if (require.main === module) {
  app.listen(app.get('port'), () => console.log('Server running on port ' + PORT))
}
module.exports = app
```

3. Checklist para otro repositorio

A. Preparar el código

1. Confirmar que la app Express se exporta y que `listen` solo corre en local (ver patrón arriba).
2. Buscar variables de entorno: `grep -rn "process.env" --include=*.js .`. Todo lo que no sea `PORT` hay que cargarlo en Vercel (Settings → Environment Variables).
3. Crear `vercel.json`, `.vercelignore` y `engines` como se muestra arriba.
4. Si el repo tiene otro nombre, actualizar `name` en `package.json` / `package-lock.json` y cualquier link o texto que mencione el nombre viejo (`grep -rn "nombre-viejo" --exclude-dir=node_modules .`).
5. Commit y push a GitHub.

B. Importar en Vercel (lo hace el usuario)

1. Entrar a **vercel.com/new** y hacer login con GitHub.
2. Si el repo no aparece: **Adjust GitHub App Permissions** y darle acceso al repo.
3. Elegir el repo → **Import**. Dejar **Application Preset: Other** y **Root Directory: ./**.
4. Cargar las variables de entorno si las hay y hacer clic en **Deploy**.
5. Si pide verificación de identidad, usar el código que llega por mail.

C. Rama de producción y rama de trabajo

1. En **Settings** → **Git** confirmar que **Production Branch = main**.
2. Crear `develop`: `git checkout -b develop` y `git push -u origin develop`.
3. Trabajar en `develop` (genera deploys de **preview**, no tocan producción). Para publicar, merge a `main`.
4. Opcional: en **Settings** → **Git** → **Ignored Build Step** poner `[ "$VERCEL_GIT_COMMIT_REF" != "main" ]` para que solo `main` despliegue.

D. Verificar

1. Abrir la URL de producción y probar `/` y un endpoint real (acá: `/api/all`, respondió 200 con datos).
2. Actualizar el README con la URL nueva y el flujo de ramas.

4. Cosas que conviene saber

> **Aviso «Possible configuration mismatch» / «Due to builds existing in your configuration file…».** Es esperable: con `builds` en `vercel.json` Vercel ignora los ajustes de build del panel. No impide el deploy. Si molesta, se puede borrar `builds` y `routes`, porque Vercel ya detecta Express con `index.js` exportando la app.

> **El `.env` no se sube.** Está en `.gitignore` y `.vercelignore`. En Vercel, `PORT` no hace falta; el resto de las variables se cargan en el panel.

> **Puerto local distinto del esperado.** Si el servidor arranca en un puerto que no es el del código, revisar el `.env` (`PORT` tiene prioridad sobre el valor por defecto).

> **Límites de Vercel.** Las funciones tienen tiempo máximo por request; si una fuente externa tarda, puede dar timeout. Logs en el panel (pestaña Logs) o con `vercel logs <url>`.

5. Resultado en este proyecto

- Repo: `github.com/andresGitDev/usd-alfam-services`
- Producción: `https://usd-alfam-services.vercel.app/` (ejemplo: `/api/all`)
- Ramas: `main` (producción, despliegue automático en cada push) y `develop` (trabajo / preview).
- Commits: `84f0e5b` configuración Vercel, `bc1fea1` renombrado, y un commit de README en `develop`.